

Project Executable Files

Date	02 July 2026
Team ID	
Project Name	Personalized Networking Assistant
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	N/A
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	N/A
7	APK / Executable binary (if applicable for mobile/desktop apps)	N/A
8	Dockerfile / Containerization config (if applicable)	N/A
9	Test files and test results	Yes

10	Demo video or walkthrough (if required)	Yes
----	---	-----

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```

Project/
├── app/
│   ├── __init__.py
│   ├── main.py                # FastAPI app entry point
│   ├── config.py              # Model names, API URLs, env vars
│   ├── models/
│   │   ├── __init__.py
│   │   └── schemas.py         # Pydantic request/response models
│   ├── routers/
│   │   ├── __init__.py
│   │   └── conversation.py    # API route handlers
│   └── services/
│       ├── __init__.py
│       ├── event_analyzer.py   # DistilBERT zero-shot theme extraction
│       ├── topic_generator.py  # GPT-2 conversation-starter generation
│       ├── fact_checker.py     # Wikipedia API fact-check
│       ├── history_logger.py   # Persist sessions → data/history.json
│       └── feedback_logger.py  # Persist ratings → data/feedback.json
├── frontend/
│   └── streamlit_app.py        # Streamlit UI (light theme)
├── tests/
│   ├── conftest.py
│   ├── test_event_analyzer.py
│   ├── test_topic_generator.py
│   ├── test_fact_checker.py
│   └── test_routes.py
├── data/
│   ├── networking_events_dataset.json # 8 sample networking events
│   ├── history.json                 # Auto-generated at runtime
│   └── feedback.json                # Auto-generated at runtime
├── .env                            # Local env vars (not committed)
├── .env.example                    # Template - copy to .env
├── requirements.txt
└── README.md
  
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://networking-assistant-bml6.onrender.com
Login Credentials (Demo)	Not Required
Platform / Hosting Provider	Render
Repository Link	https://github.com/AP24110010242/Personalized-Networking-Assistant
Demo Video Link	https://github.com/AP24110010242/Personalized-Networking-Assistant

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Prerequisites:

- Python 3.11 or above
- Git
- Virtual Environment (venv)
- FastAPI
- Streamlit
- Hugging Face Transformers
- PyTorch

Steps

Step 1:

Clone the repository

```
git clone <GitHub Repository URL>
```

Step 2:

Navigate to the project directory

```
cd personalized-networking-assistant
```

Step 3:

Create a virtual environment

```
python -m venv venv
```

Step 4:

Activate the virtual environment

Windows:

```
venv\Scripts\activate
```

Linux/macOS:

```
source venv/bin/activate
```

Step 5:

Install dependencies

```
pip install -r requirements.txt
```

Step 6:

Start the FastAPI backend

```
uvicorn app.main:app --reload
```

Step 7:

Open another terminal and start Streamlit

```
streamlit run frontend/streamlit_app.py
```

Step 8:

Open the application in your browser

<http://localhost:8501>

Step 5: Known Issues / Limitations

S. No	Known Issue / Limitation	Workaround / Status
1	Initial model loading takes time because Hugging Face models are downloaded on first execution.	Models are cached locally after the first run.
2	Wikipedia fact verification depends on internet connectivity.	Ensure an active internet connection while running the application.
3	The application currently runs locally and is not deployed online.	Deployment can be performed later using platforms such as Render or Railway.